

ds2time的解题思路

题目信息

题目名	类型	难度
ds2time	DS	简单

FLAG

- 动态flag

知识点

1. 数据识别与分类

解题步骤

1、拿到附件的data.txt后打开发现是英文文本和敏感数据混杂的数据，然后按照附件提供数据格式规范文档里的数据识别与提取规则编写以下exp.py，实现识别与分类

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
敏感数据提取脚本
从文本文件中提取手机号、银行卡号、IP地址等敏感数据
"""

import re
import sys
from typing import List, Set, Tuple

def luhn_check(card_number: str) -> bool:
    """
    Luhn算法验证银行卡号
    """
    def digits_of(n):
        return [int(d) for d in str(n)]

    digits = digits_of(card_number)
    odd_digits = digits[-1::-2]
    even_digits = digits[-2::-2]
    checksum = sum(odd_digits)
    for d in even_digits:
        checksum += sum(digits_of(d*2))
    return checksum % 10 == 0

def is_valid_phone(phone: str) -> bool:
    """
    验证手机号是否有效
    - 11位纯数字
    """
```

```

- 以1开头
- 根据数据格式规范文档，后8位可以是任意数字
"""
if not phone.isdigit() or len(phone) != 11:
    return False

if not phone.startswith('1'):
    return False

return True

def is_valid_bankcard(card: str) -> bool:
    """
    验证银行卡号是否有效
    - 13-19位纯数字
    - 首位不能为0
    """
    if not card.isdigit():
        return False

    if len(card) < 13 or len(card) > 19:
        return False

    if card.startswith('0'):
        return False

    return True

def is_valid_ip(ip: str) -> bool:
    """
    验证IP地址是否有效
    - 4个段，用点分隔
    - 每段0-255
    - 不允许前导零（除了单独的0）
    """
    parts = ip.split('.')
    if len(parts) != 4:
        return False

    for part in parts:
        if not part.isdigit():
            return False

        # 检查前导零
        if len(part) > 1 and part.startswith('0'):
            return False

        num = int(part)
        if num < 0 or num > 255:
            return False

    return True

```

```

def extract_sensitive_data(text: str) -> Tuple[Set[str], Set[str], Set[str]]:
    """
    从文本中提取敏感数据
    返回：(手机号集合, 银行卡号集合, IP地址集合)
    """
    phones = set()
    bankcards = set()
    ips = set()

    # 提取手机号 - 11位数字, 以1开头
    phone_pattern = r'\b1\d{10}\b'
    phone_matches = re.findall(phone_pattern, text)
    for phone in phone_matches:
        if is_valid_phone(phone):
            phones.add(phone)

    # 提取银行卡号 - 13-19位数字, 不以0开头
    bankcard_pattern = r'\b[1-9]\d{12,18}\b'
    # 过滤掉手机号长度的数字, 避免重复
    bankcard_matches = [m for m in re.findall(bankcard_pattern, text) if len(m) !=
11]
    for card in bankcard_matches:
        if is_valid_bankcard(card):
            bankcards.add(card)

    # 提取IP地址 - xxx.xxx.xxx.xxx格式
    ip_pattern = r'\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b'
    ip_matches = re.findall(ip_pattern, text)
    for ip in ip_matches:
        if is_valid_ip(ip):
            ips.add(ip)

    return phones, bankcards, ips

def main():
    """
    主函数
    """
    input_file = 'english_text_with_sensitive_data.txt'
    output_file = 'extracted_data.txt'

    try:
        # 读取输入文件
        with open(input_file, 'r', encoding='utf-8') as f:
            text = f.read()

        print(f"正在从 {input_file} 提取敏感数据...")

        # 提取敏感数据
        phones, bankcards, ips = extract_sensitive_data(text)

```

```

# 统计结果
total_count = len(phones) + len(bankcards) + len(ips)
print(f"提取结果统计:")
print(f"手机号: {len(phones)} 条")
print(f"银行卡号: {len(bankcards)} 条")
print(f"IP地址: {len(ips)} 条")
print(f"总计: {total_count} 条")

# 保存结果 (按照数据格式规范文档要求添加前缀)
with open(output_file, 'w', encoding='utf-8') as f:
    # 写入手机号
    for phone in sorted(phones):
        f.write(f"phone_{phone}\n")

    # 写入银行卡号
    for card in sorted(bankcards):
        f.write(f"bankcard_{card}\n")

    # 写入IP地址
    for ip in sorted(ips):
        f.write(f"ip_{ip}\n")

print(f"提取结果已保存到 {output_file}")

# 如果提取的数据少于5000条, 给出提示
if total_count < 5000:
    print(f"\n注意: 提取到的数据({total_count}条)少于预期的5000条")
    print("可能的原因:")
    print("1. 文本中的有效数据确实少于5000条")
    print("2. 验证规则过于严格, 需要调整")
    print("3. 正则表达式匹配规则需要优化")

except FileNotFoundError:
    print(f"错误: 找不到输入文件 {input_file}")
    sys.exit(1)
except Exception as e:
    print(f"错误: {str(e)}")
    sys.exit(1)

if __name__ == '__main__':
    main()

```

2、运行以上exp.py得到answer.txt, 打开看到数据已成功提取, 结果如下图所示。

phone_13004360582
phone_13004740410
phone_13005887718
phone_13008791718
phone_13013233998
phone_13016325321
phone_13023074374
phone_13028338880
phone_13028418055
phone_13039414328
phone_13040134106
phone_13040641185
phone_13041986905
phone_13045091246
phone_13046089586
phone_13048107595
phone_13050495484
phone_13057017961
phone_13057377578
phone_13057493668
phone_13061442462
phone_13067222250
phone_13067793564
phone_13070214311
phone_13073683685
phone_13081911402
phone_13082910366
phone_13102721749
phone_13103771140
phone_13105805783
phone_13108818649
phone_13110338715

3、访问验证靶机，提交answer.py，发现成功通过并且拿到flag。

文件上传区: (请在此处上传文件)



Drop file here or [click to upload](#)

只允许上传 .txt 文件格式 (其中文件名称勿随意)
[点击查看下载示例文件](#)

📄 answer.txt

结果展示区: (请注意, 刷新页面结果将清空)

文件名	上传状态	上传相关备注	分数(百分比)	FLAG
answer.txt	1	上传成功	100.000%	DASCTF{8e551a8f3959ef14c1c9eb8f1f5f68d8}